

Lecture Notes:

Convolutional Neural Networks

CIC1205 - Machine Learning

Eduardo Bezerra

May 22, 2026

0 Contents

Before You Read	3
Learning Objectives	3
1 Introduction	4
1.1 Why Not Just Use an MLP for Images?	4
1.2 The Three Key Inductive Biases	4
2 The Convolution Operation	5
2.1 Filters and Feature Maps	5
2.2 A Worked Example: 2D Convolution	6
2.3 Padding and Stride	7
2.4 Channels: Depth in Convolutional Layers	8
3 Pooling Layers	9
3.1 Purpose of Pooling	9
3.2 Max Pooling and Average Pooling	9
4 CNN Architecture	10
4.1 The Convolutional Block	10
4.2 Fully Connected Layers and the Classification Head	11
4.3 A Concrete Architecture: A LeNet-Style CNN	11
4.4 Receptive Field	13
5 Training CNNs	13
5.1 CNNs Train by Backpropagation	13
5.2 Batch Normalization	13
5.3 Data Augmentation for Images	14
5.4 Transfer Learning	15
6 Implementation in PyTorch	15
6.1 Defining a CNN	16
6.2 Data Loading and Preprocessing	17
6.3 Evaluating a Trained CNN	18
7 Common Pitfalls and Debugging CNNs	19
7.1 First Sanity Checks	19
7.2 Debugging Checklist	19
7.3 The Receptive Field and Feature Hierarchy	19
8 Notable CNN Architectures	20
8.1 From LeNet to Modern CNNs	20
8.2 Residual Connections	21
9 Connection to the Broader Deep Learning Landscape	23
9.1 CNNs as Structured MLPs	23
9.2 Vision Transformers	23

10 Final Remarks	24
10.1 Summary of Key Concepts	24
10.2 Recommended Next Steps	24

0 Before You Read

These notes build directly upon the MLP lecture notes. The following concepts are assumed to be already understood. Review them if necessary before proceeding.

- The MLP architecture: layers, neurons, weight matrices, and bias vectors.
- The forward pass and the backward pass (backpropagation).
- Activation functions (ReLU in particular) and loss functions.
- Gradient descent and the general training loop.
- Overfitting and basic regularization strategies.
- Basic familiarity with images as 2D or 3D arrays of pixel values.

0 Learning Objectives

After studying these notes, you should be able to:

- explain why fully connected networks are poorly suited for image data, and articulate the three inductive biases that CNNs exploit;
- describe the convolution operation and compute the output of a convolutional layer given an input, a kernel, a stride, and a padding value;
- determine the spatial dimensions of feature maps after convolution and pooling operations using the standard dimension formulas;
- explain the role of pooling layers and their effect on spatial resolution and translation invariance;
- describe the typical block structure of a CNN (convolution $\rightarrow$ activation $\rightarrow$ pooling) and identify its components in a concrete architecture;
- trace the flow of data through a LeNet-style architecture for an image classification task;
- implement a basic CNN for image classification in PyTorch;
- diagnose common CNN training problems and apply appropriate fixes.

1 Introduction

1.1 Why Not Just Use an MLP for Images?

The MLP studied in the previous lectures is a powerful and general model: given enough neurons and layers, it can approximate any continuous function. One might therefore expect that applying an MLP directly to raw image pixels would be sufficient for visual recognition tasks. In practice, this approach fails for three interrelated reasons.

Parameter explosion. A modest image of size 224×224 with three color channels contains $224 \times 224 \times 3 = 150,592$ pixel values. If the first hidden layer has only 512 neurons, the first weight matrix alone has $150,592 \times 512 \approx 77$ million parameters. A network deep enough to learn useful visual abstractions would require billions of parameters, making training infeasible without enormous datasets.

No spatial structure is exploited. An MLP treats the input as a flat vector. All spatial relationships between pixels are destroyed the moment the image is unrolled. The network must re-learn from scratch that nearby pixels are more likely to be related than distant ones. This is an enormous amount of structure to re-discover from data alone.

No translation invariance. A fully connected layer trained to detect a cat in the top-left corner of an image will not automatically detect the same cat in the bottom-right corner. The weight patterns that identify the feature in one location are entirely separate from those at other locations. Visual recognition should, intuitively, be independent of where an object appears in the image.

Convolutional Neural Network

A Convolutional Neural Network (CNN) is a feedforward neural network specifically designed for data with a grid-like topology, most commonly images. It replaces fully connected layers with **convolutional layers** that exploit three structural properties of visual data: *local connectivity*, *parameter sharing*, and *translation equivariance*. These properties make CNNs far more efficient and effective than MLPs for image recognition tasks.

1.2 The Three Key Inductive Biases

CNNs are effective because they encode assumptions (inductive biases) that match the structure of natural images.

- **Local connectivity.** Meaningful visual patterns such as edges, textures, and shapes are *local*: a pixel is primarily related to its immediate neighbors, not to pixels on the other side of the image. Convolutional layers therefore connect each neuron to only a small *receptive field* in the preceding layer, rather than to all neurons.
- **Parameter sharing.** The same local pattern detector (a learned filter, also called a *kernel*) is applied at every spatial location of the input. A filter that detects a

horizontal edge will detect it whether it appears at the top, middle, or bottom of the image. This sharing reduces the number of parameters dramatically.

- **Translation equivariance.** Because the same kernel is applied everywhere, shifting the input by a few pixels shifts the output feature map by the same amount. Concretely, a convolutional layer detects the same pattern at different positions using the same kernel, but the corresponding activation appears at the correspondingly shifted position in the feature map. It does *not* produce an identical output regardless of position. This property is called *translation equivariance*. Approximate *translation invariance* (where position is genuinely ignored) emerges later, through pooling, global aggregation, and the classification head.

Historical context. The idea of using local filters for image recognition was pioneered by LeCun et al. [1998], whose LeNet-5 architecture successfully applied CNNs to handwritten digit recognition in the late 1990s. The modern deep learning era for vision began in 2012, when AlexNet [Krizhevsky et al., 2012] dramatically outperformed all other methods on the ImageNet competition by training a deep CNN on GPUs. Since then, CNNs have become the dominant approach for a wide range of visual tasks.

Entry Ticket 1

Think about what you already know about MLPs before reading about convolutions.

1. Consider an MLP applied to a 32×32 grayscale image (a single channel, i.e., 1024 pixel values) with a single hidden layer of 512 neurons. How many parameters are in the first weight matrix? Now compare this to a CNN where the first layer has 32 filters of size 3×3 applied to the same single-channel input. How many parameters does the CNN layer have? (For reference, CIFAR-10 images are 32×32 but have *three* RGB channels, which would further increase the MLP count to $32 \times 32 \times 3 \times 512 = 1,572,864$ weights, while the CNN layer would need only $32 \times (3 \times 3 \times 3 + 1) = 896$ parameters.)
2. Informally, what do you think it means for a filter to “slide” over an image? How is this different from what an MLP weight matrix does?
3. Why might the same edge-detection filter be useful in many different parts of an image? What assumption about images does this reflect?

2 The Convolution Operation

2.1 Filters and Feature Maps

The fundamental operation in a CNN is the *discrete convolution* (more precisely, cross-correlation) between an input and a learnable *kernel* (also called a *filter*). The kernel is a small matrix of weights, typically of size $k \times k$ where $k \in \{3, 5, 7\}$. The kernel is slid across the input, and at each position, the element-wise product between the kernel and the overlapping input region is summed, producing a single output value. The result of sliding the kernel across the entire input is called a *feature map* (or *activation map*).

Discrete Convolution (Cross-Correlation)

Given a 2D input $\mathbf{X} \in \mathbb{R}^{H \times W}$ and a kernel $\mathbf{K} \in \mathbb{R}^{k \times k}$, the output feature map $\mathbf{Z} \in \mathbb{R}^{H' \times W'}$ is computed as:

$$Z_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X_{i+m, j+n} \cdot K_{m,n} + b,$$

where b is a learned bias term and (i, j) indexes the output spatial position. This operation is applied for all valid (i, j) .

Convolution vs. Cross-Correlation

In signal processing, convolution involves flipping the kernel before applying it. Deep learning frameworks implement *cross-correlation*, which does not flip the kernel. Because the kernel is learned, this distinction has no practical consequence: a flipped kernel is just another learnable kernel. The term “convolution” is used throughout this field to mean cross-correlation.

2.2 A Worked Example: 2D Convolution

Consider a 4×4 input and a 3×3 kernel (no padding, stride 1):

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 3 & 2 \\ 2 & 0 & 1 & 1 \\ 1 & 3 & 2 & 0 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}.$$

The output size is $(4 - 3 + 1) \times (4 - 3 + 1) = 2 \times 2$. Computing each output element:

$$\begin{aligned} Z_{0,0} &= (1)(1) + (2)(0) + (0)(-1) + (0)(1) + (1)(0) + (3)(-1) + (2)(1) + (0)(0) + (1)(-1) \\ &= 1 + 0 + 0 + 0 + 0 - 3 + 2 + 0 - 1 = -1, \end{aligned}$$

$$\begin{aligned} Z_{0,1} &= (2)(1) + (0)(0) + (1)(-1) + (1)(1) + (3)(0) + (2)(-1) + (0)(1) + (1)(0) + (1)(-1) \\ &= 2 + 0 - 1 + 1 + 0 - 2 + 0 + 0 - 1 = -1, \end{aligned}$$

$$\begin{aligned} Z_{1,0} &= (0)(1) + (1)(0) + (3)(-1) + (2)(1) + (0)(0) + (1)(-1) + (1)(1) + (3)(0) + (2)(-1) \\ &= 0 + 0 - 3 + 2 + 0 - 1 + 1 + 0 - 2 = -3, \end{aligned}$$

$$\begin{aligned} Z_{1,1} &= (1)(1) + (3)(0) + (2)(-1) + (0)(1) + (1)(0) + (1)(-1) + (3)(1) + (2)(0) + (0)(-1) \\ &= 1 + 0 - 2 + 0 + 0 - 1 + 3 + 0 + 0 = 1. \end{aligned}$$

The output feature map is:

$$\mathbf{Z} = \begin{bmatrix} -1 & -1 \\ -3 & 1 \end{bmatrix}.$$

The kernel used above is a vertical edge detector: it responds positively where intensity decreases from left to right and negatively where it increases. In this worked example, the kernel is chosen by hand so that we can interpret its effect. In a real CNN, however, the entries of the kernel are trainable parameters initialized randomly and adjusted by backpropagation; the network is not given hand-crafted filters such as Sobel or Laplacian operators.

Figure 1 illustrates the sliding window mechanism. The kernel is positioned at one location of the input, an element-wise product is computed, and the sum produces one output value. The kernel then shifts by one position (stride = 1) and the operation is repeated.

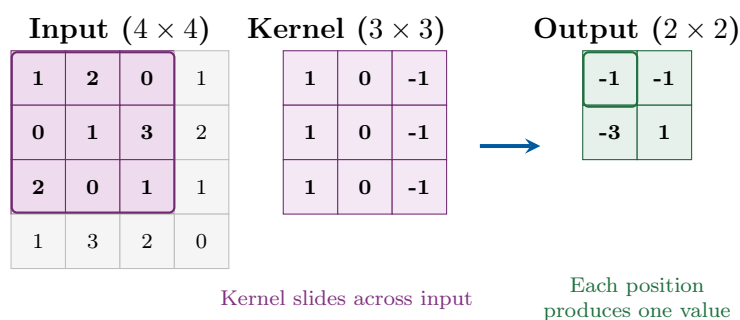


Figure 1: The convolution operation. The 3×3 kernel is applied to each overlapping 3×3 region of the 4×4 input (highlighted), computing an element-wise product and summing to produce a single output value. Sliding the kernel across all valid positions produces the 2×2 feature map.

2.3 Padding and Stride

Two hyperparameters control the spatial extent and resolution of the output.

Padding

By default, convolution reduces the spatial dimensions of the input. To preserve the original height and width, *zero-padding* adds rows and columns of zeros around the border of the input before applying the kernel. With *same* padding ($p = \lfloor k/2 \rfloor$ for an odd kernel size k), the output has the same height and width as the input.

Stride

The *stride* s controls how many pixels the kernel moves at each step. A stride of 1 produces the densest possible output. A stride of 2 skips every other position and halves the spatial dimensions of the output. Larger strides are sometimes used as a fast downsampling alternative to pooling.

Output size formula

Given an input of height H (or width W), kernel size k , padding p , and stride s , the output spatial dimension is:

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1.$$

Table 1 shows examples of how different parameter choices affect the output size for a 32×32 input.

Table 1: Output spatial dimensions for a 32×32 input under different convolution settings.

Kernel k	Padding p	Stride s	Output size	Effect
3	0	1	30	Slight shrink
3	1	1	32	Preserved (same padding)
3	0	2	15	Downsampling
5	2	1	32	Preserved
7	3	2	16	Aggressive downsampling

2.4 Channels: Depth in Convolutional Layers

The definition above describes the single-channel case. For RGB images or intermediate CNN layers, the same idea generalizes to tensors with multiple channels: the kernel has one spatial slice per input channel, and the results are summed across both space and channels to produce a single 2D feature map.

Formally, a convolutional layer receives a 3D input tensor of shape (C_{in}, H, W) , where C_{in} is the number of input channels. The kernel must also be 3D, with shape (C_{in}, k, k) , so that it spans all input channels. The operation computes the element-wise product across all channels simultaneously and sums to produce one 2D feature map.

To detect multiple types of features, a convolutional layer uses C_{out} different kernels (one per output channel). The output is therefore a tensor of shape $(C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$, where each channel contains the feature map produced by one kernel.

The full kernel tensor for a layer has shape $(C_{\text{out}}, C_{\text{in}}, k, k)$, containing $C_{\text{out}} \times C_{\text{in}} \times k \times k$ learnable weights, plus C_{out} bias terms.

Counting Parameters in Convolutional Layers

A convolutional layer with C_{in} input channels, C_{out} output channels (filters), and kernel size $k \times k$ has:

$$C_{\text{out}} \times (C_{\text{in}} \times k \times k + 1) \quad \text{learnable parameters.}$$

For example, suppose the input has shape $(3, 32, 32)$, as in an RGB image, and the layer has 64 filters of size 3×3 . Each filter has shape $(3, 3, 3)$ because it spans all three input channels. Each filter independently produces one output feature map of

shape $(H_{\text{out}}, W_{\text{out}})$. Therefore, 64 filters produce an output tensor with 64 channels and the total number of learnable parameters is $64 \times (3 \times 3 \times 3 + 1) = 64 \times 28 = 1,792$ — far fewer than the millions a fully connected layer would require for the same spatial extent.

Entry Ticket 2

Work through the following before reading about pooling.

1. An input feature map has shape $(16, 28, 28)$ (16 channels, 28×28 spatial dimensions). A convolutional layer applies 32 filters of size 3×3 with padding 1 and stride 1. What is the shape of the output? How many parameters does this layer have?
2. What would the output shape be if the same layer used stride 2 instead of stride 1?
3. What is the benefit of using many small (3×3) kernels stacked in multiple layers, compared to one large (7×7) kernel in a single layer?

3 Pooling Layers

3.1 Purpose of Pooling

Convolutional layers produce feature maps that retain high spatial resolution. As the network deepens, it becomes computationally expensive and also undesirable to maintain full resolution: a small shift in the input should not change the detection of a high-level feature. Pooling layers reduce the spatial dimensions of the feature maps, decreasing computational cost and introducing a degree of translation invariance.

Pooling Layer

A **pooling layer** applies a fixed aggregation function over non-overlapping (or partially overlapping) windows of the input feature map, producing a smaller output. The operation has no learnable parameters. Common choices are max pooling and average pooling.

3.2 Max Pooling and Average Pooling

Max pooling takes the maximum value within each window. For a 2×2 max pooling with stride 2 applied to a feature map of size $H \times W$, the spatial dimensions are typically halved when H and W are even; more generally, the same output-size formula used for convolution applies, replacing the kernel size by the pooling window size. Max pooling preserves the most activated feature in each region, acting as a local invariance mechanism.

Average pooling takes the mean value within each window. It produces smoother outputs and is sometimes used at the end of the network (global average pooling) as an alternative to flattening.

Worked Example

$$\text{Input: } \begin{bmatrix} 3 & 1 & 2 & 4 \\ 0 & 5 & 1 & 2 \\ 2 & 3 & 0 & 1 \\ 1 & 0 & 4 & 3 \end{bmatrix} \xrightarrow{\text{max pool } 2 \times 2, s=2} \begin{bmatrix} 5 & 4 \\ 3 & 4 \end{bmatrix}.$$

The 2×2 windows are $\{3, 1, 0, 5\}$, $\{2, 4, 1, 2\}$, $\{2, 3, 1, 0\}$, and $\{0, 1, 4, 3\}$; their maxima are 5, 4, 3, 4.

Pooling vs. Strided Convolution

Modern architectures sometimes replace pooling layers with strided convolutions ($s = 2$), which achieve similar downsampling while keeping the operation learnable. In practice, both approaches appear widely; max pooling remains common due to its simplicity and its ability to preserve sharp activations.

Figure 2 illustrates max pooling and average pooling side by side for the same 4×4 input.

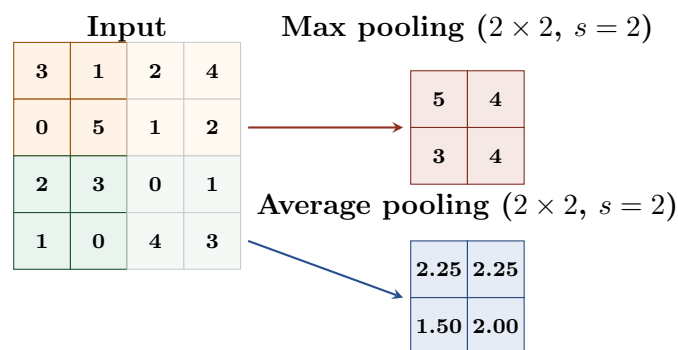


Figure 2: Max pooling and average pooling, both with a 2×2 window and stride 2 applied to the same 4×4 input. The two outputs are shown in the same column: max pooling above and average pooling below.

Common Misconception

Pooling does not affect the number of channels. A feature map of shape (C, H, W) after a 2×2 max pooling with stride 2 becomes $(C, H/2, W/2)$ when H and W are even: the spatial dimensions are halved, but the channel count is unchanged. Only convolutional layers change the number of channels.

4 CNN Architecture

4.1 The Convolutional Block

The typical building block of a CNN is the *convolutional block*, which consists of:

1. A *convolutional layer* that applies learnable kernels to produce feature maps.
2. An *activation function* (typically ReLU) applied element-wise to the feature maps.

3. Optionally, a *batch normalization* layer (discussed in Section 5.2).
4. A *pooling layer* to reduce spatial dimensions.

Several such blocks are stacked in sequence. The spatial dimensions decrease and the number of channels typically increases as we go deeper, following the intuition that the network gradually converts spatial information into increasingly abstract feature representations.

4.2 Fully Connected Layers and the Classification Head

After the convolutional blocks, the feature maps are *flattened* into a 1D vector and passed through one or more fully connected (dense) layers. This *classification head* takes the high-level features extracted by the convolutional blocks and maps them to the final output (class probabilities for classification, or a scalar for regression).

An alternative to flattening is *global average pooling* (GAP), which computes the spatial average of each feature map, producing a vector of length equal to the number of channels. GAP reduces the number of parameters in the classification head and helps regularize the model.

4.3 A Concrete Architecture: A LeNet-Style CNN

LeCun et al. [1998] introduced a CNN for handwritten digit recognition (MNIST). Table 2 describes a LeNet-style architecture layer by layer, adapted to modern conventions. Although small by contemporary standards, this architecture exemplifies all the core concepts of CNN design.

Shape Tracing

When implementing a CNN, always track tensors in the form (C, H, W) for a single example or (N, C, H, W) for a mini-batch. Most shape errors arise from forgetting the channel dimension or from miscomputing the spatial size before the first fully connected layer. A convenient way to verify is to run a single dummy input through the network and print the shape at each step.

For the architecture in Table 2, starting from $(1, 28, 28)$:

$$\begin{aligned}
 (1, 28, 28) &\xrightarrow{\text{Conv1, pad}=2} (6, 28, 28) \xrightarrow{\text{Pool1}} (6, 14, 14) \\
 &\xrightarrow{\text{Conv2, no pad}} (16, 10, 10) \xrightarrow{\text{Pool2}} (16, 5, 5) \\
 &\xrightarrow{\text{Flatten}} 400.
 \end{aligned}$$

The value 400 is $16 \times 5 \times 5$, which determines the input size of the first fully connected layer.

Figure 3 illustrates the data flow through this architecture. The key observation is that spatial resolution decreases (from 28×28 to 14×14 to 5×5) while the depth (number of channels) increases (from 1 to 6 to 16), before the feature maps are flattened and passed to the fully connected layers.

Table 2: LeNet-style architecture adapted to 28×28 grayscale inputs (MNIST-like), using modern choices such as ReLU and max pooling.

Layer	Type	Output shape	Parameters
Input	—	(1, 28, 28)	0
Conv1	Conv 5×5 , $C_{\text{out}} = 6$, pad=2	(6, 28, 28)	156
ReLU	Activation	(6, 28, 28)	0
Pool1	MaxPool 2×2 , $s = 2$	(6, 14, 14)	0
Conv2	Conv 5×5 , $C_{\text{out}} = 16$, no pad	(16, 10, 10)	2,416
ReLU	Activation	(16, 10, 10)	0
Pool2	MaxPool 2×2 , $s = 2$	(16, 5, 5)	0
Flatten	—	(400,)	0
FC1	Linear $400 \rightarrow 120$	(120,)	48,120
FC2	Linear $120 \rightarrow 84$	(84,)	10,164
FC3 (output)	Linear $84 \rightarrow 10$	(10,)	850
Total			61,706

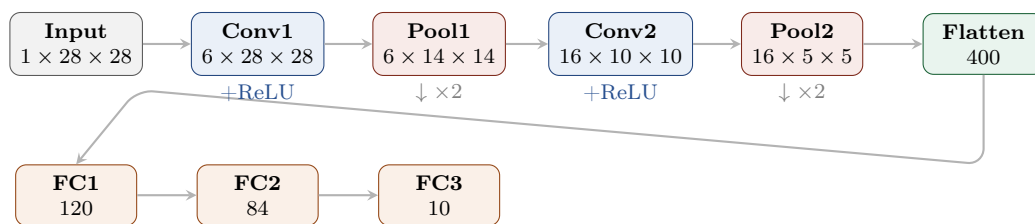


Figure 3: Data flow through a LeNet-style CNN applied to 28×28 grayscale images. Convolutional and pooling layers progressively reduce spatial resolution while increasing depth. The fully connected layers map the flattened feature vector to class probabilities.

Entry Ticket 3

Work through the LeNet-style architecture in Table 2.

1. Verify the number of parameters for Conv1 and Conv2 using the formula from Section 2.4.
2. The FC1 layer has $400 \times 120 + 120 = 48,120$ parameters. Even in this small network, the fully connected layers dominate the parameter count. What does this suggest about the relative roles of convolutional and fully connected parts?
3. If you doubled the number of filters in both Conv1 (to 12) and Conv2 (to 32), how would the total parameter count change? Which layers would be most affected?

4.4 Receptive Field

The *receptive field* of a neuron in a feature map is the region in the original input that can influence its value. A single 3×3 convolution has a receptive field of 3×3 . After two stacked 3×3 convolutions, the receptive field grows to 5×5 ; after three, it becomes 7×7 . Pooling and strided convolutions further increase the receptive field.

This is a key reason why deep networks with small kernels are preferred over shallow networks with large kernels: stacking multiple 3×3 layers achieves a large receptive field with fewer parameters and more non-linearity. For instance, two 3×3 convolutional layers have $2 \times (9C^2) = 18C^2$ parameters (for C channels) versus one 5×5 layer with $25C^2$ parameters, while achieving the same 5×5 receptive field with more non-linearity.

Why Stack Small Kernels

Two stacked 3×3 convolutional layers have the same receptive field as one 5×5 layer but use $\frac{18C^2}{25C^2} = 72\%$ of the parameters and apply the non-linearity twice instead of once. Three stacked 3×3 layers match a 7×7 receptive field using only $\frac{27C^2}{49C^2} \approx 55\%$ of the parameters. Modern architectures (VGG, ResNet) use almost exclusively 3×3 kernels for this reason.

5 Training CNNs

5.1 CNNs Train by Backpropagation

Despite their different structure, CNNs are trained using exactly the same principle as MLPs: forward pass, loss computation, backpropagation, and parameter update. The convolution operation is differentiable, so the chain rule applies straightforwardly. Deep learning frameworks (PyTorch, TensorFlow) handle this automatically through their autograd engines, so practitioners do not need to implement convolution gradients manually.

Same Training Principle

The training of a CNN does not require any new mathematical concept beyond what was covered in the MLP notes. Backpropagation applies the chain rule to a computation graph, and the convolution operation is simply another differentiable function in that graph.

The one structural detail worth noting is that each kernel weight contributes to *many* output positions, because the same weight is reused as the kernel slides over the input. During backpropagation, the gradient for a shared kernel weight is therefore obtained by accumulating contributions from all spatial locations where that weight was used. This is handled transparently by the autograd engine.

5.2 Batch Normalization

Batch normalization [Ioffe and Szegedy, 2015] is a normalization layer inserted after a convolutional (or linear) layer, typically before the activation function. It normalizes the pre-activations within a mini-batch to have zero mean and unit variance, then applies a learned scale (γ) and shift (β):

$$\hat{z}_i = \frac{z_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad \tilde{z}_i = \gamma \hat{z}_i + \beta,$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the batch mean and variance. During inference, running statistics accumulated during training are used instead of batch statistics.

Batch Normalization in Convolutional Layers

In convolutional layers, batch normalization is applied *channel-wise*: for each output channel, the mean and variance are computed over the mini-batch *and* over all spatial positions. The learned parameters γ and β are also defined per channel. This is why `nn.BatchNorm2d(C)` in PyTorch creates exactly $2C$ learnable parameters (one γ and one β per channel), regardless of the spatial size $H \times W$.

The most common ordering in introductory CNNs is $\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU}$, although some modern architectures use pre-activation variants ($\text{ReLU} \rightarrow \text{Conv} \rightarrow \text{BN}$) in their residual blocks.

Batch normalization provides several practical benefits:

- It stabilizes training by keeping activations in a well-conditioned range, reducing sensitivity to initialization and learning rate.
- It acts as a mild regularizer, reducing the need for dropout in convolutional architectures.
- It allows the use of higher learning rates, speeding up convergence.

5.3 Data Augmentation for Images

Because CNNs are trained on finite datasets, data augmentation is a powerful and essential regularization technique in the image domain. Label-preserving transformations are applied randomly during training, effectively increasing the diversity of the training set.

Common augmentation strategies include:

- *Random horizontal flip*: reflects the image left-to-right. Appropriate for natural images; inappropriate for tasks where orientation matters (e.g., digit recognition, medical imaging).
- *Random crop*: randomly extracts a patch of fixed size from a slightly larger padded image, introducing positional variation.
- *Color jitter*: randomly perturbs brightness, contrast, saturation, and hue.
- *Random rotation*: rotates the image by a small angle.
- *Cutout / Random erasing*: masks out a random rectangular region of the image, forcing the network to use context rather than a single discriminative region.

Augmentation Is Applied Only During Training

Data augmentation is applied to training examples only. Validation and test images should always be processed with the same, fixed preprocessing (typically center-crop

and normalization). Applying random augmentation to the test set would introduce variance into the evaluation results.

5.4 Transfer Learning

Training a CNN from scratch requires large datasets and significant computation. In practice, it is common to start from a model that has already been trained on a large dataset (such as ImageNet) and *fine-tune* it on the target task. This is called transfer learning.

The rationale is that low-level features (edges, textures, color gradients) are common to most visual tasks, so the early convolutional layers of a pre-trained model provide a strong initialization for the new task. Two common strategies are:

- *Feature extraction*: freeze all convolutional layers, replace only the classification head, and train the head on the new data.
- *Fine-tuning*: initialize with pre-trained weights, then train the entire network (or just the later layers) on the new data with a low learning rate.

A Practical Transfer Learning Strategy

A common practical approach is to begin with *feature extraction*: train only the new classification head while keeping the convolutional base frozen. If performance plateaus, one can then unfreeze the later convolutional blocks and fine-tune them with a smaller learning rate. Early layers are often kept frozen when the target dataset is small, since they encode general low-level features (edges, textures) that are useful across tasks. The more the target domain differs from the source (e.g., medical images vs. natural photos), the more layers may need to be fine-tuned.

Transfer learning is especially powerful when the target dataset is small, as it allows leveraging millions of training examples from the source domain.

Entry Ticket 4

Consider a CNN being trained on a medical imaging dataset with only 1,000 labeled images.

1. Would you train a CNN from scratch or use transfer learning? Justify your choice.
2. Which layers of a pre-trained ImageNet model would you expect to be most directly reusable for a medical imaging task? Which layers might require more adaptation?
3. Name two data augmentation strategies that would be appropriate for chest X-ray classification, and two that might not be appropriate. Explain your reasoning.

6 Implementation in PyTorch

6.1 Defining a CNN

Listing 1 implements a LeNet-style CNN for 10-class image classification in PyTorch, designed for MNIST-like grayscale inputs of shape (1,28,28). The structure mirrors the architecture in Table 2, with the addition of batch normalization and dropout for modern training practices.

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 class LeNetCNN(nn.Module):
6     def __init__(self, num_classes=10):
7         super().__init__()
8
9         # Convolutional feature extractor
10        self.features = nn.Sequential(
11            # Block 1: (1,28,28) -> (6,28,28) -> (6,14,14)
12            nn.Conv2d(in_channels=1, out_channels=6,
13                    kernel_size=5, padding=2),
14            nn.BatchNorm2d(6),
15            nn.ReLU(),
16            nn.MaxPool2d(kernel_size=2, stride=2),
17
18            # Block 2: (6,14,14) -> (16,10,10) -> (16,5,5)
19            nn.Conv2d(in_channels=6, out_channels=16,
20                    kernel_size=5),
21            nn.BatchNorm2d(16),
22            nn.ReLU(),
23            nn.MaxPool2d(kernel_size=2, stride=2),
24        )
25
26        # Classification head: (16,5,5) -> 400 -> 120 -> 84 -> 10
27        self.classifier = nn.Sequential(
28            nn.Flatten(), # (16,5,5) ->
29                (400,)
30            nn.Linear(16 * 5 * 5, 120),
31            nn.ReLU(),
32            nn.Dropout(p=0.3),
33            nn.Linear(120, 84),
34            nn.ReLU(),
35            nn.Linear(84, num_classes), # logits (no
36                softmax)
37        )
38
39        def forward(self, x):
40            x = self.features(x)
41            x = self.classifier(x)
42            return x
43
44        # --- Instantiate and train ---
45        model = LeNetCNN(num_classes=10)

```

```

44 criterion = nn.CrossEntropyLoss()           # expects logits (see
    note below)
45 optimizer = optim.Adam(model.parameters(), lr=1e-3)
46
47 for epoch in range(20):
48     model.train()
49     for images, labels in train_loader:
50         optimizer.zero_grad()
51         outputs = model(images)               # forward pass
52         loss = criterion(outputs, labels)
53         loss.backward()                       # backpropagation
54         optimizer.step()                     # parameter update

```

Listing 1: LeNet-style CNN for 10-class grayscale image classification (MNIST) in PyTorch.

Logits, Softmax, and Cross-Entropy

In PyTorch, `nn.CrossEntropyLoss()` expects **raw logits** (the unnormalized scores output by the last linear layer), not probabilities. The model should therefore **not** end with a `Softmax` layer during training. Internally, `CrossEntropyLoss` combines `LogSoftmax` and the negative log-likelihood loss in a single, numerically stable operation. A common bug is to add `Softmax` to the model and then pass probabilities to `CrossEntropyLoss`, which produces silently incorrect gradients.

6.2 Data Loading and Preprocessing

A complete training pipeline in PyTorch uses the `Dataset` and `DataLoader` abstractions for efficient data loading, and `torchvision.transforms` for preprocessing and augmentation. Listing 2 shows how to set up data pipelines for training and validation on MNIST, applying augmentation only to the training set.

```

1 from torchvision import datasets, transforms
2 from torch.utils.data import DataLoader
3
4 # Training transform: augmentation + normalization
5 train_transform = transforms.Compose([
6     transforms.RandomHorizontalFlip(),
7     transforms.RandomCrop(28, padding=2),
8     transforms.ToTensor(),
9     # MNIST channel statistics (single channel)
10    transforms.Normalize(mean=(0.1307,), std=(0.3081,)),
11 ])
12
13 # Validation transform: normalization only (no augmentation)
14 val_transform = transforms.Compose([
15     transforms.ToTensor(),
16     transforms.Normalize(mean=(0.1307,), std=(0.3081,)),
17 ])
18

```

```

19 train_dataset = datasets.MNIST(root='./data', train=True,
20                               download=True, transform=
21                               train_transform)
22 val_dataset    = datasets.MNIST(root='./data', train=False,
23                               download=True, transform=
24                               val_transform)
25
26 train_loader = DataLoader(train_dataset, batch_size=64, shuffle=
27                           True,
28                           num_workers=2)
29 val_loader    = DataLoader(val_dataset, batch_size=64, shuffle=
30                           False,
31                           num_workers=2)

```

Listing 2: Data loading with preprocessing and augmentation for MNIST (training and validation).

Always Normalize Inputs

As with MLPs, input normalization is essential for CNNs. Pixel values in $[0, 255]$ should be scaled to $[0, 1]$ (via `ToTensor()`) and then standardized to zero mean and unit variance using channel-wise statistics computed from the training set. Using the wrong normalization statistics, or omitting normalization entirely, commonly causes slow convergence or failure to train.

6.3 Evaluating a Trained CNN

Listing 3 shows how to evaluate the model on the validation set. Batch normalization and dropout behave differently during training and evaluation, so `model.eval()` must be called before evaluating, and gradients should be disabled with `torch.no_grad()` to save memory.

```

1 model.eval()
2 correct, total = 0, 0
3
4 with torch.no_grad():
5     # disable gradient
6     computation
7     for images, labels in val_loader:
8         outputs = model(images)
9         predictions = outputs.argmax(dim=1)
10        correct += (predictions == labels).sum().item()
11        total   += labels.size(0)
12
13 accuracy = correct / total
14 print(f"Validation accuracy: {accuracy:.3f}")

```

Listing 3: Evaluation loop for a trained CNN.

Common Misconception

Forgetting to call `model.eval()` before evaluation is one of the most common bugs in PyTorch. Batch normalization layers use different statistics during training (batch statistics) versus evaluation (running statistics), and dropout is active during training but disabled during evaluation. Evaluating in training mode will produce incorrect and unreliable results, especially when the batch size is small.

7 Common Pitfalls and Debugging CNNs

7.1 First Sanity Checks

Before fine-tuning hyperparameters, always verify the following:

- *Shape consistency.* Print the output shape of each layer (using `torchsummary` or by running a dummy input) before training. Mismatched dimensions cause errors that are hard to debug.
- *Can the model overfit one batch?* Train for many epochs on a single mini-batch of 4–8 images. The training loss should reach near-zero. If not, the architecture, loss function, or learning rate has a fundamental problem.
- *Input normalization.* Confirm that pixel values are normalized and that the mean/std used for normalization come from the training set.
- *Train and evaluation modes.* Ensure `model.train()` and `model.eval()` are called at the correct points in the training loop.
- *Label correctness.* Verify that `CrossEntropyLoss` receives integer class indices, not one-hot vectors.

7.2 Debugging Checklist

Table 3 summarizes common symptoms and their likely causes when training CNNs.

7.3 The Receptive Field and Feature Hierarchy

A useful mental model for understanding what a CNN learns is the *feature hierarchy*:

- Early layers (small receptive field): detect low-level features such as oriented edges, blobs, and color gradients.
- Middle layers (medium receptive field): detect mid-level patterns such as corners, textures, and simple shapes by combining low-level features.
- Late layers (large receptive field): detect high-level, task-specific features such as object parts and whole objects.

This hierarchy emerges automatically from training, without any explicit supervision about what each layer should detect. Understanding this hierarchy is useful for debugging: if a CNN trained on a new task performs poorly, visualizing early-layer activations can reveal whether the feature extractor is learning sensible representations.

Table 3: Common symptoms, likely causes, and fixes when training CNNs.

Symptom	Likely cause	Fix
Loss NaN from the start	LR too large, missing normalization	Reduce LR; normalize inputs
Loss does not decrease	LR too small; <code>eval()</code> called during training	Increase LR; check <code>train()</code> call
Low train loss, high val loss	Overfitting	Data augmentation, dropout, reduce model
High train and val loss	Underfitting	Deeper/wider architecture, more epochs
Accuracy stuck near $1/K$	Labels incorrect or wrong loss	Check integer labels, verify loss function
Slow training	Batch size too small; no GPU	Increase batch size; use GPU
Features not generalizing	Too little data; no augmentation	Transfer learning; add augmentation
<code>eval()</code> accuracy lower than expected	<code>model.train()</code> not reset to <code>eval()</code>	Always call <code>model.eval()</code> before evaluation
<code>RuntimeError</code> on first forward pass	Channel mismatch (e.g., model expects 1 channel, input has 3)	Check <code>in_channels</code> matches dataset

Entry Ticket 5

A CNN trained on MNIST achieves 99% training accuracy but only 80% validation accuracy. You observe that the validation loss begins to rise after epoch 15.

1. What phenomenon is occurring? Name it precisely and explain it in terms of what the model is doing.
2. Describe three concrete interventions from this set of notes that could help, and explain the mechanism behind each one.
3. If you apply dropout with $p = 0.5$ after every convolutional layer, would you expect this to help as much as in an MLP? Why might dropout be less standard after convolutional layers than after fully connected layers?

8 Notable CNN Architectures

8.1 From LeNet to Modern CNNs

The field of CNN architecture design has evolved rapidly since LeNet. Table 4 summarizes the most influential architectures and their key innovations. Understanding this evolu-

tion is useful not merely as historical context, but because each architectural innovation addresses a specific problem that arises when training deeper and wider networks.

Table 4: Evolution of influential CNN architectures, their key innovations, and impact.

Architecture	Year	Key innovation	Significance
LeNet-5	1998	Local convolutions, pooling, classification head	Established the CNN template
AlexNet	2012	Deep CNN on GPU, ReLU, dropout, data augmentation	Revived deep learning; ImageNet breakthrough
VGGNet	2014	All 3×3 kernels, very deep (16–19 layers)	Showed that depth matters; simple and powerful
GoogLeNet	2014	Inception modules (parallel multi-scale convolutions)	Efficient wide networks without parameter explosion
ResNet	2015	Residual connections (skip connections)	Enabled training of very deep networks (50–152+ layers)
DenseNet	2017	Dense connections (each layer connects to all later layers)	Maximum feature reuse, strong regularization
MobileNet	2017	Depthwise separable convolutions	Efficient CNNs for mobile/embedded deployment
EfficientNet	2019	Compound scaling of width, depth, and resolution	Systematic scaling for high accuracy and efficiency

8.2 Residual Connections

The most significant architectural innovation after AlexNet is the *residual connection* (or *skip connection*) introduced in ResNet [He et al., 2016]. A residual block computes:

$$\mathbf{u}^{(l+2)} = \mathcal{F}(\mathbf{a}^{(l)}) + \mathbf{a}^{(l)}, \quad \mathbf{a}^{(l+2)} = \text{ReLU}(\mathbf{u}^{(l+2)}),$$

where $\mathcal{F}$ represents two (or more) stacked convolutional layers and $\mathbf{a}^{(l)}$ is the identity shortcut bypassing them.

Residual connections address the *degradation problem*: beyond a certain depth, plain networks can become harder to optimize and may even show *higher training error* than shallower counterparts, even in the absence of overfitting. This difficulty is partly related to vanishing gradients, but also to the fundamental challenge of learning identity-like transformations through many stacked nonlinear layers. Skip connections make identity mappings easy to represent, since the network only needs to learn the **residual** (the deviation from identity) rather than the full transformation.

Why Residual Connections Work

In a residual block, the gradient of the loss with respect to an early layer's output contains a direct additive term from the skip connection:

$$\frac{\partial L}{\partial \mathbf{a}^{(l)}} = \frac{\partial L}{\partial \mathbf{a}^{(l+2)}} \left(I + \frac{\partial \mathcal{F}}{\partial \mathbf{a}^{(l)}} \right).$$

The identity matrix I provides a direct gradient component, so that even if $\partial \mathcal{F} / \partial \mathbf{a}^{(l)}$ is very small, the gradient can still propagate through the skip connection more easily than in a plain deep network. ResNets with over 1,000 layers have been trained successfully using this principle.

Figure 4 illustrates the structure of a residual block.

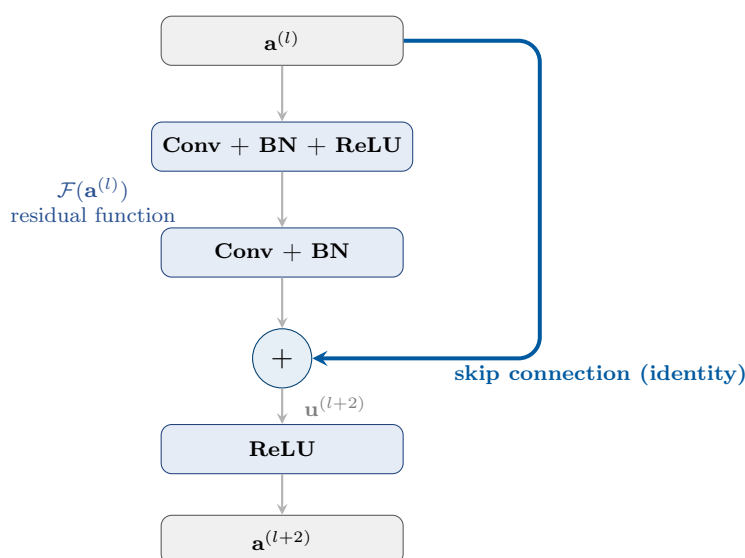


Figure 4: A residual block. The output of the convolutional branch $\mathcal{F}(\mathbf{a}^{(l)})$ is summed with the identity shortcut $\mathbf{a}^{(l)}$ to form the pre-activation $\mathbf{u}^{(l+2)}$, which is then passed through ReLU to produce $\mathbf{a}^{(l+2)}$. The skip connection allows gradients to flow directly from $\mathbf{a}^{(l+2)}$ back to $\mathbf{a}^{(l)}$, mitigating optimization difficulties in deep networks.

CNNs Beyond Image Classification

The CNN architecture described in these notes is used for image classification, but the same components appear in many other tasks:

- *Object detection* (e.g., YOLO, Faster R-CNN): predicts bounding boxes and class labels for multiple objects in one image.
- *Semantic segmentation* (e.g., U-Net, DeepLab): assigns a class label to every pixel.
- *Image generation* (e.g., GANs, Diffusion models): uses convolutional layers in the generator and discriminator.
- *Medical imaging*: CNNs are widely applied for tumor detection, X-ray analysis, and pathology slide classification.

In all these tasks, the same building blocks (convolutional layers, pooling, batch normalization, skip connections) appear; only the network head and loss function

are adapted to the task.

9 Connection to the Broader Deep Learning Landscape

9.1 CNNs as Structured MLPs

A CNN can be understood as an MLP with a very specific constraint on the weight matrices. In other words, a convolutional layer can be imagined as a very large fully connected layer in which most connections are removed (local connectivity) and many remaining weights are tied together (parameter sharing). This is why CNNs have far fewer free parameters than MLPs applied directly to flattened images, and why they generalize better from limited data.

More formally, this constraint is a form of *structural regularization*: it reduces the effective number of free parameters, makes the model more sample-efficient, and encodes prior knowledge about the structure of visual data.

The general principle is important: many modern architectures can be seen as MLPs with structured constraints that reflect the geometry of the input data. CNNs exploit the grid structure of images; Recurrent Neural Networks exploit the sequential structure of language; Graph Neural Networks exploit the graph structure of molecules or social networks; Transformers exploit the set-pairwise relationship structure of tokens.

9.2 Vision Transformers

Since 2020, Vision Transformers (ViT) [Dosovitskiy et al., 2021] have challenged the dominance of CNNs in image recognition. A ViT divides an image into fixed-size patches, treats each patch as a token, and applies a standard Transformer encoder. Remarkably, ViTs can match or exceed CNN performance when trained on very large datasets, without any convolution.

This does not render CNNs obsolete. CNN-based architectures remain competitive, especially when data is limited or when inference efficiency matters. Modern architectures often combine convolutional layers with attention mechanisms (e.g., ConvNeXt, EfficientViT). The key lesson is that inductive biases (local connectivity, parameter sharing) are helpful when data is scarce, but may be unnecessary when data and compute are abundant.

Entry Ticket 6

Reflect on the connections between CNNs and the broader course content.

1. A CNN is sometimes described as a “structured MLP.” What specific structures are imposed on the weight matrices? What benefit does each structure provide?
2. Backpropagation through a convolutional layer is more complex than through a fully connected layer, yet the training principle is the same. What property of the convolution operation makes this possible?
3. Without using any CNN-specific vocabulary, describe in one sentence what a

trained CNN learns to do. How does this relate to the concept of *representation learning* introduced in the MLP notes?

10 Final Remarks

10.1 Summary of Key Concepts

The following points summarize the essential content of these notes:

- MLPs are poorly suited for images due to parameter explosion, lack of spatial structure exploitation, and the absence of translation invariance.
- CNNs address these limitations through **local connectivity** (kernels span a small spatial region), **parameter sharing** (the same kernel is applied at every position), and **translation equivariance** (shifted input produces a correspondingly shifted feature map). Approximate translation invariance emerges through pooling and the classification head.
- The **convolution operation** slides a learned kernel across the input, computing a dot product at each location to produce a **feature map**. The output size is determined by input size, kernel size, padding, and stride.
- A convolutional layer with C_{out} filters of size $C_{\text{in}} \times k \times k$ has $C_{\text{out}} \times (C_{\text{in}} \times k^2 + 1)$ parameters.
- **Pooling layers** reduce spatial dimensions without learnable parameters, introducing local translation invariance.
- The standard CNN architecture stacks convolutional blocks (Conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Pool), followed by a fully connected classification head.
- CNNs are trained by **backpropagation** and gradient descent, exactly as MLPs. Because kernel weights are shared across spatial positions, their gradients are accumulated over all spatial positions where each weight contributed.
- **Batch normalization** stabilizes training and reduces sensitivity to initialization; in CNNs it is applied channel-wise. **Data augmentation** is the primary regularization strategy in the image domain.
- **Transfer learning** from pre-trained models is commonly used when labeled data is scarce, and often involves first training only the classification head, then optionally fine-tuning later convolutional blocks.
- **Residual connections** enable training of very deep networks by providing direct gradient paths and by making identity mappings easy to learn.
- The CNN is the standard foundation for image classification, object detection, segmentation, and many other vision tasks.

10.2 Recommended Next Steps

These notes prepare the student for the following topics in the deep learning module:

MLP → **CNNs** → Sequence Models (RNN/LSTM) → Attention & Transformers → Generative Models

Key Takeaways

- CNNs exploit the spatial structure of images through local connectivity and parameter sharing, achieving far greater efficiency than MLPs on visual data.
- The convolution operation is the core building block: a learned kernel slides across the input to produce a feature map.
- The same training principles (forward pass, backpropagation, gradient descent) apply to CNNs as to MLPs; the gradient of each shared weight is accumulated over all positions where it was used.
- Batch normalization (applied channel-wise in CNNs) and data augmentation are standard tools for stable and generalizable CNN training.
- Modern CNNs are deep and make heavy use of residual connections. When labeled data is limited, initializing from pre-trained weights is a common and effective strategy.
- Understanding CNNs thoroughly is a prerequisite for all modern vision architectures, including Vision Transformers.

10 References

- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 448–456, 2015.
- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, 2012.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.